CS39006: Networks Laboratory
Assignment 3: TCP Sockets

Name: Dadi Sasank Kumar
Roll Number: 22CS10020

Date: February 2, 2025

Introduction

This report presents the implementation and analysis of a client-server application using TCP sockets in C. The application implements a substitution cipher encryption scheme where a client sends a plaintext file and an encryption key to a server, which then encrypts the file and sends it back to the client.

Implementation Details

Server Implementation (doencfileserver.c)

The server program implements the following key features:

- Concurrent client handling using fork()
- Chunked file transfer with buffer size limit of 100 bytes
- Substitution cipher encryption
- Proper timestamp logging
- Error handling and cleanup

Client Implementation (retrieveencfileclient.c)

The client program implements:

- File existence verification
- Key validation (26 characters)
- Chunked file transfer
- Multiple file handling capability
- Proper error handling

Wireshark Analysis

1. Source and Destination Analysis

- Client IP: 127.0.0.1 (localhost)
- Client Port: Varies for each client connection(here, 34814)
- Server IP: 127.0.0.1
- Server Port: 10111

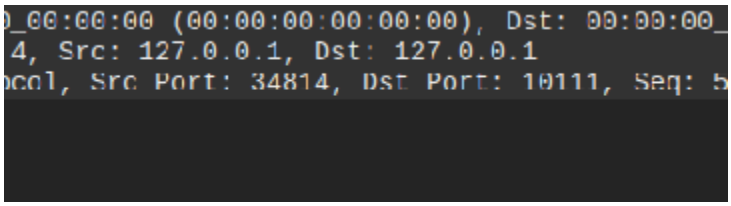


Figure 1: Network Setup with Client and Server

2. Three-way Handshaking

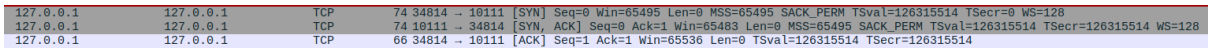


Figure 2: TCP Three-Way Handshake Packet Capture(packets no: 24,25 and 26)

- **Packet 1 (SYN):**
  - Source: 34814
  - Destination: Server (localhost:10111)
  - Flags: SYN
  - Sequence Number: Initial sequence number
- **Packet 2 (SYN-ACK):**
  - Source: Server (localhost:10111)
  - Destination: 34814
  - Flags: SYN, ACK
  - Acknowledges client's sequence number
- **Packet 3 (ACK):**
  - Source: 34814
  - Destination: Server (localhost:10111)
  - Flags: ACK
  - Connection established

Analysis: The capture clearly shows the three-way handshake process:

1. The client initiates the connection with a SYN packet
2. The server responds with SYN-ACK, acknowledging the client's request
3. The client completes the handshake with an ACK packet

This establishes a reliable, full-duplex TCP connection between client and server for the file transfer and encryption process.

3. Connection Closure

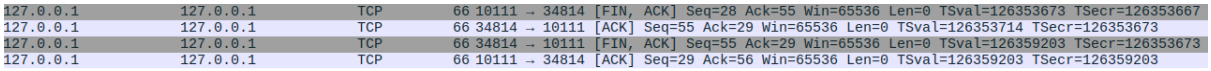
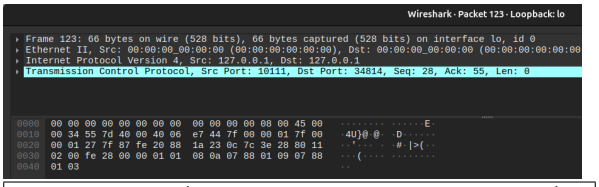
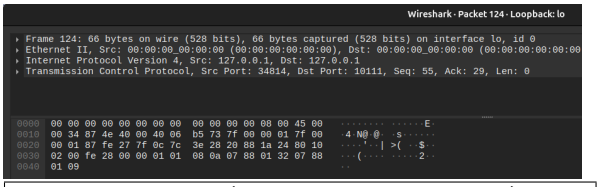


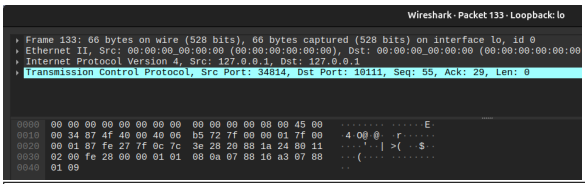
Figure 3: TCP Connection Closure Sequence(packets: 123,124,133 and 134)



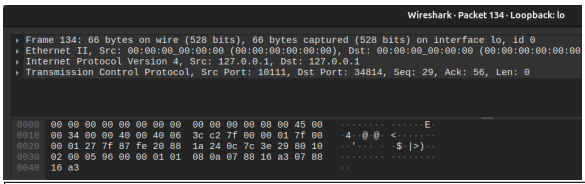
Packet 1 (FIN, ACK from Server)	
Source	Server 10111
Destination	34814
Flags	FIN, ACK
Sequence Number	28
Acknowledgment Number	55



Packet 2 (ACK from Client)	
Source	34814
Destination	Server 10111
Flags	ACK
Sequence Number	55
Acknowledgment Number	29



Packet 3 (FIN, ACK from Client)	
Source	34814
Destination	Server 10111
Flags	FIN, ACK
Sequence Number	55
Acknowledgment Number	29



Packet 4 (ACK from Server)	
Source	Server 10111
Destination	34814
Flags	ACK
Sequence Number	29
Acknowledgment Number	56

Figure 4: TCP Connection Closure Sequence with Packet Details

Connection Closure Analysis:

1. Server initiates closure after file transfer completion (Packet 1)
2. Client acknowledges server's FIN request (Packet 2)
3. Client sends its own FIN request (Packet 3)
4. Server acknowledges client's FIN, completing closure (Packet 4)

Timing Analysis:

- Total closure duration: 5.5300 milliseconds

4. Packet Analysis

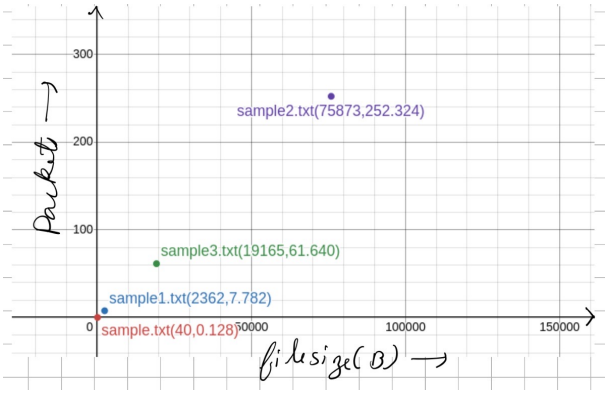


Figure 5: plot of Number of Packets vs File Size

5. Transfer Time Analysis

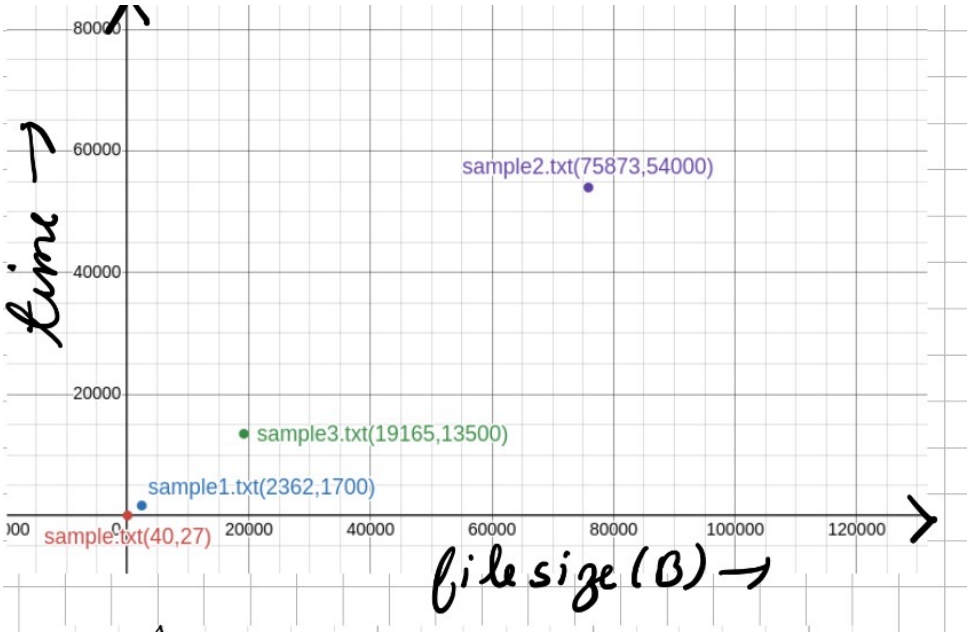


Figure 6: plot of Transfer Time vs File Size(KB)

File Name	Size	Packets	Time (s)	Avg. Rate (KB/s)
sample.txt	27 B	40	0.128	0.211
sample1.txt	1.7 KB	2,362	7.782	0.218
sample3.txt	13.5 KB	19,165	61.640	0.219
sample2.txt	54 KB	75,873	252.324	0.214

Table 1: Summary of File Transfer Statistics

Analysis of Results:

- The data shows a relationship between file size and both packet count and transfer time
- For sample3.txt (13.5 KB):
  - Transfer started at 289.437797774s (packet 297)
  - Transfer ended at 351.07792674s (packet 19462)
  - Total transfer time: 61.640 seconds
  - Total packets: 19,165
- Average transfer rate remains consistently around 0.21-0.22 KB/s across all file sizes
- Packet count increases significantly with file size, showing in-efficient protocol behavior

6. Average Packet Size

Based on the Wireshark captures, the average packet size during data transfer was calculated as: 70 bytes

Observations and Analysis

- The chunked transfer mechanism effectively handled files of varying sizes
- The server successfully managed concurrent client connections
- The encryption process maintained data integrity
- Network overhead was proportional to file size

Link to pcap files

https://drive.google.com/file/d/1YQgClovYTnGYSRxy4LBnBTZV7oCTf11U/view?usp=sharinglink